Laboratory No. 9: Transformer and GPT

Attention head, multi-head attention, block, GPT language model (training in Lab 10)

Lab 9: Transformer / GPT

Faculty of Mathematics and Information Science
Warsaw University of Technology

April 20, 2026

Laboratory Rules

- All places you need to implement something are highlighted
- To check each part: run `python ./part_1.py`, `python ./part_2.py`, `python ./part_3.py`
- On this lab please use chat

What You Will Do

1. **Part 1** (`part_1.py`): FeedForward and AttentionHead (single self-attention with Q,K,V).
2. **Part 2** (`part_2.py`): MultiHeadAttention and Block (concat heads, project; LayerNorm + residual).
3. **Part 3** (`part_3.py`): GPT-style language model (config, embeddings, blocks, forward, generate, init, count_parameters).

What You Will Do

Part 1: Q, K, V as Linear Layers

We get query, key, value from the same input x by three linear projections:

- `self.key = nn.Linear(n_embd, head_size, bias=False)` $\Rightarrow K = xW_K$
- `self.query = nn.Linear(n_embd, head_size, bias=False)` $\Rightarrow Q = xW_Q$
- `self.value = nn.Linear(n_embd, head_size, bias=False)` $\Rightarrow V = xW_V$

Then attention weights and output:

$$\mathrm{wei} = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right), \qquad \mathrm{out} = \mathrm{wei} \cdot V$$

Part 1: Forward - Q, K, V

- k = self.key(x) (B, T, head_size)
- q = self.query(x) (B, T, head_size)
- v = self.value(x) (B, T, head_size)

q has shape: batch size, sequence length, and head size.
To compute attention scores, we multiply q with k.
k needs to be transposed:
$k^\top = (B, head_size, T)$
Now:
q @ $k^\top \rightarrow (B, T, T)$
This gives a score for every token attending to every token.

Part 1: Forward - Scaling

- The raw attention scores: [q @ $k^\top \to (B, T, T)$]

- These values can become large, which makes softmax unstable (exploding/vanishing gradients).
- To stabilize training, we scale the scores: [scaled_scores = q @ $k^\top \cdot \frac{1}{\sqrt{\text{head_size}}}$]

  In code:

  - scaled_attention_scores = q @ k.transpose(-2, -1) * (k.shape[-1] ** -0.5)

Part 1: Forward - Causal Mask

The model must not look at future tokens.

We create a lower-triangular mask: $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$

This ensures:

- token 0 sees only token 0
- token 1 sees tokens 0,1
- token 2 sees tokens 0,1,2

In code:

- scaled_attention_scores.masked_fill(self.tril[:T, :T] == 0, float("-inf"))

After softmax, masked positions get probability 0.

Part 1: Example of Attention Calculation

First, we compute raw attention scores: [q @ $k^\top = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 2 & 2 \\ 3 & 1 & 2 \end{bmatrix}$]

Apply causal mask (future tokens not allowed): [$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} 2 & -\infty & -\infty \\ 1 & 2 & -\infty \\ 3 & 1 & 2 \end{bmatrix}$]

Explanation: wherever mask $= 0$, we replace the value with $-\infty$.

Now apply softmax: [$\begin{bmatrix} 2 & -\infty & -\infty \\ 1 & 2 & -\infty \\ 3 & 1 & 2 \end{bmatrix} ; \xrightarrow{\text{softmax}} ; \begin{bmatrix} 1 & 0 & 0 \\ 0.27 & 0.73 & 0 \\ 0.66 & 0.09 & 0.24 \end{bmatrix}$]

Each row sums to 1:

- shows how much each token attends to allowed (past) tokens

Part 1: Forward - Weighted Sum

Finally, we combine information using values:

- out = attention_weights @ v

Shapes:

- attention_weights: (B, T, T)
- v: (B, T, head_size)
- output: (B, T, head_size)

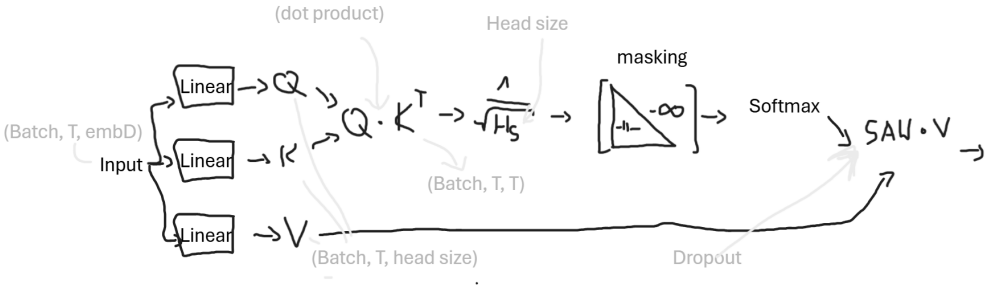
Each token becomes a weighted sum of other tokens' value vectors. This is the final output of one attention head.

Part 1: Example

Attention weights: $\begin{bmatrix} 1 & 0 & 0 \\ 0.27 & 0.73 & 0 \\ 0.66 & 0.09 & 0.24 \end{bmatrix}$ $\quad$ v $= \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$ out = attention_weights @ v =

$\begin{bmatrix} 1 & 0 \\ 0.27 & 0.73 \\ 0.90 & 0.33 \end{bmatrix}$ Each token becomes a weighted sum of previous tokens' value vectors.

What You Will Do



Part 1: Attention Head - Pseudocode

Input: $x \in \mathbb{R}^{B \times T \times C}$

Step 1: Linear projections

- $q = W_q x$
- $k = W_k x$
- $v = W_v x$

Step 2: Compute attention scores [scores $=$ q $\cdot k^\top / \sqrt{d_k}$]

Step 3: Apply causal mask

- scores[i,j] $= -\infty$ if $j > i$

Step 4: Normalize [weights $=$ softmax(scores)] [weights $=$ dropout(weights)]

Step 5: Weighted sum [out $=$ weights $\cdot v$]

Output: $out \in \mathbb{R}^{B \times T \times d_k}$

Part 1: Help Info

1. **register_buffer('tril', ...)** – Registers a tensor that is not a parameter but is saved with the model; use for the causal mask so it stays on the right device.
2. **masked_fill(mask, value)** – Where `mask` is True, fill with `value` (e.g. `float('-inf')` so softmax gives 0).
3. **torch.tril(ones(T,T))** – Lower triangular matrix of ones; `tril[:T,:T]` for variable length.
4. **F.softmax(scores, dim=-1)** – Softmax over the last dimension.
5. **k.shape[-1]**-0.5** – Scaling factor $1/\sqrt{d_k}$ for attention scores.

Part 2: What is Multi-Head Attention?

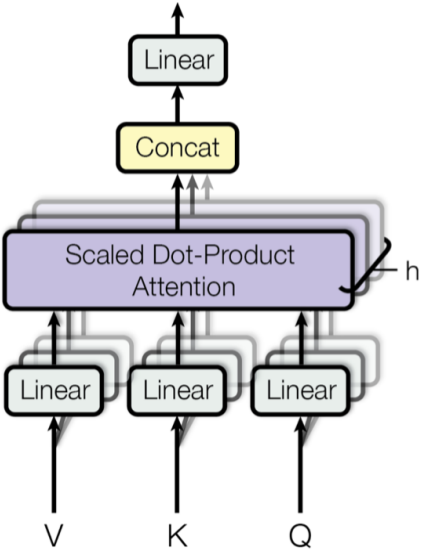
Instead of using only one attention head, we use several heads in parallel.

- Each head gets the same input x
- Each head learns different attention patterns
- Their outputs are concatenated and projected back to *n_embd*

In code:

- `self.heads = nn.ModuleList([AttentionHead(...) for _ in range(num_heads)])`
- `self.proj = nn.Linear(head_size * num_heads, n_embd)`

Multi-Head Attention Diagram



Part 2: Multi-Head Attention - Parallel Heads

Each attention head processes the same input:

- head 1: $h_1(x)$
- head 2: $h_2(x)$
- ...
- head n: $h_n(x)$

If each head outputs:

$$(B, T, head_size)$$

then each head learns its own representation of the sequence.
Example intuition:

- one head may focus on nearby tokens
- another head may focus on long-range dependencies
- another head may focus on punctuation or structure

Part 2: Forward - Concatenation

In the forward pass:

- out = torch.cat([h(x) for h in self.heads], dim=-1)

Suppose we have 3 heads, and each head outputs:

$$(B, T, head_size)$$

Concatenating along the last dimension gives:

$$(B, T, 3 \cdot head_size)$$

In general:

$$(B, T, num_heads \cdot head_size)$$

This combines all head outputs into one tensor.

Part 2: Example of Concatenation

Suppose we have 2 heads and each produces:

$$h_1(x) = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad h_2(x) = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Concatenating along the last dimension:

$$\text{cat}(h_1, h_2) = \begin{bmatrix} 1 & 2 & 5 & 6 \\ 3 & 4 & 7 & 8 \end{bmatrix}$$

So the features from different heads are placed side by side.

Part 2: Projection Back to Embedding Size

After concatenation, the output dimension is:

$$num_heads \cdot head_size$$

But the transformer expects dimension:

$$n_embd$$

So we use a linear layer:

- `self.proj = nn.Linear(head_size * num_heads, config.n_embd)`

In forward:

- out = self.proj(out)

This mixes information from all heads and returns to the original embedding size.

Part 2: Dropout in Multi-Head Attention

After projection, dropout is applied:

- out = self.dropout(self.proj(out))

Why?

- helps regularization
- reduces overfitting
- makes training more robust

Final output shape:

$$(B, T, n_embd)$$

Part 2: Full Multi-Head Attention Flow

Multi-head attention does:

1. apply several attention heads in parallel
2. concatenate their outputs
3. project back to *n_embd*
4. apply dropout

In formula form:

$$\text{MultiHead}(x) = \text{Proj}(\text{Concat}(head_1(x), head_2(x), ..., head_n(x)))$$

Output shape:

$$(B, T, n_embd)$$

Part 2: What is a Transformer Block?

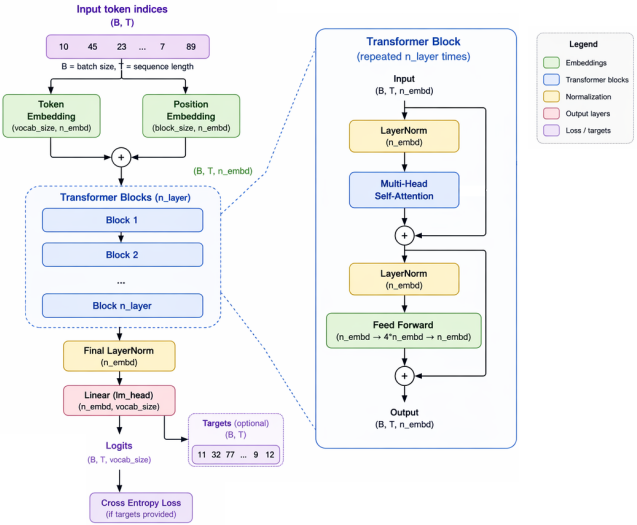
A transformer block combines:

- LayerNorm
- Multi-Head Attention
- residual connection
- LayerNorm
- FeedForward network
- residual connection

In code:

- `x = x + self.sa(self.ln1(x))`
- `x = x + self.ffwd(self.ln2(x))`

Block Diagram



Part 2: Help Info

1. **nn.ModuleList([...])** – Holds submodules; use for the list of `AttentionHeads`.
2. **torch.cat(list, dim=-1)** – Concatenate head outputs on the channel dimension.
3. **nn.LayerNorm(n_embd)** – Normalizes over the last dimension; use before attention and before feed-forward (pre-norm).

Part 3: GPTLanguageModel - Main Idea

This class defines the full GPT model.
Main components:

- token embeddings
- positional embeddings
- stack of transformer blocks
- final LayerNorm
- linear projection to vocabulary logits

In code:

- `self.token_embedding_table`
- `self.position_embedding_table`
- `self.blocks`
- `self.ln_f`
- `self.lm_head`

Part 3: Stack of Transformer Blocks

After embeddings:

$$x = \texttt{self.blocks(x)}$$

In code:

- `self.blocks = nn.Sequential(*[Block(config) for _ in range(config.n_layer)])`

This means:

- the same type of block is repeated `n_layer` times
- each block refines the token representations

If `n_layer = 6`, then input passes through 6 transformer blocks.

Part 3: Language Modeling Head

After final normalization:

$$logits = \texttt{self.lm_head(x)}$$

In code:

- `self.lm_head = nn.Linear(config.n_embd, config.vocab_size)`

This maps each token representation to vocabulary scores.
Shape:

$$(B, T, n_embd) \rightarrow (B, T, vocab_size)$$

For each position, the model predicts scores for all possible next tokens.

Part 3: Forward Shapes

Suppose input is:

$$idx \in (B, T)$$

Then the shapes are:

- token embeddings: (B, T, n_embd)
- positional embeddings: (T, n_embd)
- summed input: (B, T, n_embd)
- after blocks: (B, T, n_embd)
- logits: $(B, T, vocab_size)$

Part 3: Generate Function - Main Idea

The `generate()` function creates new tokens one by one.
Steps:

1. take the current sequence
2. crop it to the last `block_size` tokens
3. run the model
4. keep logits for the last position
5. apply softmax
6. sample the next token
7. append it to the sequence

Part 3: Cropping the Context

In code:

- `idx_cond = idx[:, -self.config.block_size:]`

Why?

- the model can only attend to at most `block_size` tokens
- if the sequence is longer, we keep only the newest part

Example:

$$idx = [4, 7, 2, 9, 5, 1]$$

if `block_size = 4`, then:

$$idx_cond = [2, 9, 5, 1]$$

Part 3: Predicting the Next Token

After the forward pass, the model returns logits for **every position**:

$$logits \in (B, T, vocab_size)$$

This means:

- for token 1, predict the next token
- for token 2, predict the next token
- ...
- for token T, predict the next token

But during generation, we only want to produce **one new token now**.
So we keep only the last position:

```
logits = logits[:, -1, :]
```

New shape:

$$(B, vocab_size)$$

This gives the prediction for **the next token after the current sequence**.

Part 3: Help Info

1. **self.apply(fn)** – Applies `fn` to every submodule and `self`; use for `_init_weights`.
2. **forward:** Use `torch.arange(T, device=idx.device)` for position indices when looking up position embeddings.
3. **torch.multinomial(probs, num_samples=1)** – Samples indices from the categorical distribution (for next-token sampling).
4. **idx[:, -block_size:]** – Last `block_size` tokens; use as context for the next step in `generate`.